

品質と
開発速度の
両立

✓ 高品質なテスト生成のための
手順・コンテキスト設計・検証法

第

特集

2 AIが書いたコード、 どうテストする？



第1章 テストを検証し
開発段階から品質を保つ

Mutation Testing
で見抜く
AIテストコードの
落とし穴

Author
渋谷 拓真

P.66

第2章 QAの現場で挑戦! AIとの
協働で高品質なテスト

**AI駆動開発の
テスト戦略**

Author
木下 智弘

P.72

第3章 従来のテストを理解し、
適切にAIに任せよう

**プロダクト品質を
保証するための
テストとAI活用**

Author
村穂 紀成

P.82

ソフトウェア開発にAIが活用され、これまでとは桁違いの速さで実装が進むようになりました。その一方で、AIが書いたコードの品質確保が課題となっています。レビューやテストにAIを利用する取り組みも活発ですが、プロダクトとしての品質保証はAI任せで大丈夫なのでしょうか？

また、人がテストし品質をチェックする場合は、テストがボトルネックにならないでしょうか？

まだ確立したノウハウのない分野ですが、本特集では、単体テスト、統合テスト、E2Eテストの各工程で、現在有効と思われる「AIが書いたコードやソフトウェアの品質を検証するための手順、手法、技術」を取り上げます。

AIによる自律的な開発ではユニットテストのコードもAIが書くことが多く、実質的にソフトウェアの品質はほとんどAIに委ねられています。人間はE2Eテストのような後のテスト工程で確認するだけでよいのでしょうか? 本章では、開発工程でも品質を確保する一手法として、テストコードを検証する方法を紹介します。

第1章

Mutation Testingで見抜く
AIテストコードの落とし穴

テストを検証し開発段階から
品質を保つ

Author 渋谷 拓真(しげや たくま)

Go Conferenceメインオーガナイザー、Kubernetesメンテナー、
Argo CDメンテナー

GitHub sivchari

AI時代の開発とテストの役割

✓ AIがコードを書く時代、
テストの責任は誰が持つのか

チーム開発では、CI (Continuous Integration、継続的インテグレーション) の成功はレビュー依頼の前提条件です。「CIが通ったのでレビューお願いします」「CIが失敗しているので直してください」。こうしたやりとりは多くの現場で日常的に行われており、CIはチーム開発の最低限の約束事として機能しています。GitHub ActionsでGoのテストを行う場合は、リスト1のような最低限の設定を行うことでCIを簡単に構築できます。

近年の開発ではAIエージェントの登場を皮切りに、AIがコードを書き、AIがユニットテストを書くプロジェクトも増加しています。し

かし、人間が書こうがAIが書こうが、コードの品質を最終的に担保するガードレールはCIであり、その中核にあるのはテストです。テストの責任は書き手ではなく、テストそのものの品質が担っているのです。

✓ CIはAIエージェントの判断基準になる

AIエージェントがコードを生成する場面が増えるにつれ、CIの位置づけは「人間のためのチェック」から「AIエージェント自身の判断基準」へと拡大しています。

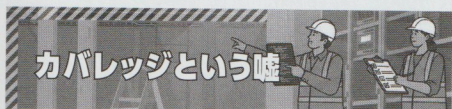
たとえば、CIの成功をゴールとして、AIエージェントに機能実装を依頼すると、エージェントはコードを書いた後にCIを監視し、その結果をもとに次の行動を決定できます(図1)。「CIが失敗したので原因を調査して修正します」「CIがすべてPASSしたのでタスクを完了します」。つまりCIは、AIエージェントが自己回帰的に

▼リスト1 最小限のGoのテストのCI設定(.github/workflows/go.yaml)

```
name: Go
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@de0fac2e4500dabe0009e67214ff5f5447ce83dd # v6.0.2
        with:
          fetch-depth: 0
      - uses: actions/setup-go@4a3601121dd01d1626a1e23e37211e3254c1c06c # v6.4.0
        with:
          go-version-file: 'go.mod'
      - run: go test ./...
```


開発を進める際のチェックポイントとして機能します。

ここで問題になるのが、CIに含まれるテストの品質です。テストが「実行されるだけで何も検証しない」ものであっても、AIエージェントは「CIが通った=正しい」と判断してしまいます。出力されるコードが指数関数的に増える中で、テストの品質が低いままCIを通過させ続けられれば、品質の低いコードが積み上がっていくことになります。テストの量だけでなく、テストそのものの品質がAIエージェントが生成するコードの品質を担保するうえで非常に重要なことになります。



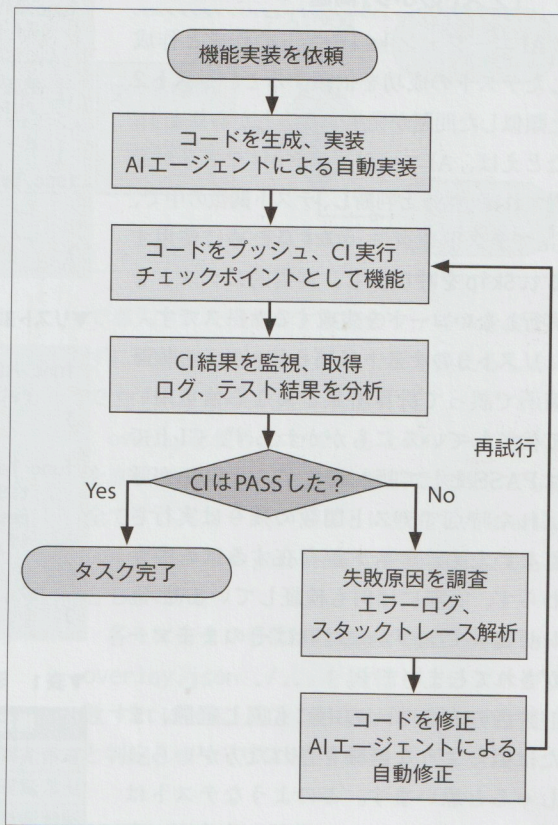
カバレッジという嘘

☑ カバレッジ100%でもバグは潜む

「ある指標が目標になると、その時点でその指標は“良い指標”ではなくなる」。これはGoodhart's Law (グッドハートの法則)^{注1}として知られる経験則です。たとえば「タイピング速度が速いほど開発生産性が高い」という仮説を立て、タイピング速度をKPI (Key Performance Indicator、重要業績評価指標) にしたらどうなるでしょうか。開発者はタイピング速度そのものに固執^{かたいり}し始め、本来の目的である生産性向上からは乖離^{かいり}していきます。「pull requestの粒度は小さいほうがレビューしやすい」という知見も、pull requestの数を貢献指標にしてしまうと、意味のない分割が目的化します。

テストカバレッジにも同じことが起こります。「カバレッジを常に80%以上に維持する」というルールを設けると、本質的に必要のないテストケースが増えていきます。数値を達成すること自体が目的になり、テストの質が置き去りに

▼図1 CIはAIエージェントが自己回帰的に開発を進める際のチェックポイント



されるのです。カバレッジは「コードのどの行がテストで実行されたか」を示す指標です。Goでは`go test -cover ./...`で簡単に計測できます。しかし、「実行された」とこと「正しく検証された」ことは別の話です。リスト2を見てみましょう。

Divは引数aとbを用いて割り算をする関数であり、TestDivはDiv関数を実際にテストするテストコードです。このテストコードに対して`go test -cover ./...`を実行すると、テストコードの中でDiv関数のすべての行を実行しているため、カバレッジは100%になります。しかし、戻り値をいっさい検証していません。Div(2, 2)が4を返しても、このテストは通ります。カバレッジは「テストされていない箇所を発見する」には有用ですが、「テストによる検証が十分か」を保証するものではありません。

注1) Charles Goodhart氏が1975年に提唱した経済学の法則。ソフトウェア開発の文脈でも広く引用されます。

✓ AIが書いたテストが引き起こす 「テストのふり」問題

AIエージェントにテストの作成と作成したテストの成功を依頼すると、リスト2と類似した問題が発生することがあります。たとえば、AIエージェントが「テストが通過すればいい」と判断し、テスト関数の中で、Goでテストをスキップするために使用するt.Skipを呼び出し、実質的にテストを実行しないコードを生成するケースです。

リスト3のテストは戻り値resultの検証箇所です。誤って計算結果と異なる値を用いて検証しているにもかかわらず、CI上ではPASSとして扱われます。t.Skipが呼ばれた時点でテスト関数の残りは実行されないため、テストが存在するにもかかわらず、実際には何も検証していません。レビューで気づかなければそのままマージされてしまいます。

読者のみなさんの中にも同じ経験、または似たような経験をされた方がいらっしゃると思います。このようなテストはすべてカバレッジには貢献しますが、実際のバグを検出する能力を持ちません。

品質の担保に意味のないテストに気づき、マージされないよう未然に防げるかどうかAIエージェントを使用するうえで重要になります。次節からテストそのものの品質を検証するMutation Testingという手法を紹介します。

Mutation Testingとは何か

✓ テストをテストするという発想

Mutation Testingは、プログラムの一部を意図的に書き換える（ミュータントを生成し）テストを実行します。プログラムの一部を意図的に書き換えることからMutation Testingでは失敗することを目的としてテストを検証します。ユニットテストを始めとしたシステム上で動く

▼リスト2 カバレッジ100%だが検証していないテスト

```
func Div(a, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("b must not be zero")
    }
    return a / b, nil
}

func TestDiv(t *testing.T) {
    _, _ = Div(2, 2)
    _, _ = Div(1, 0)
}
```

▼リスト3 AIが生成したt.Skipを含むテスト

```
func Add(a, b int) int {
    return a + b
}

func TestComplexCalc(t *testing.T) {
    t.Skip("TODO: implement this test")
    result := Add(10, 20)
    if result != 100 {
        t.Errorf("got %d, want 30", result)
    }
}
```

▼表1 ミュータントの種類と書き換え例

種類	元の演算子	書き換え後	例
算術演算子	+	-, *, /	a + b → a - b
比較演算子	==	!=	x == 0 → x != 0
論理演算子	&&		a && b → a b
ビット演算子	>>	<<	a >> b → a << b

コードを検証することを目的とした一般的なテストとは異なり、Mutation Testingは「テストをテストする」ことが目的です。

ミュータントの生成では、表1のような書き換えが行われます。

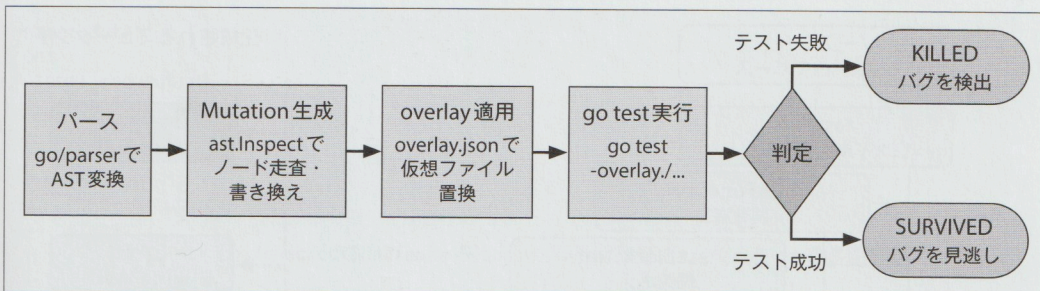
テストが失敗すればそのミュータントはKILLED（テストがバグを検出できた）、テストが成功すればミュータントはSURVIVED（テストがバグを見逃した）と判定します。いかに多くのミュータントをKILLEDできるかが、テストの品質を示す指標となります。

✓ ミュータントの生成から KILLED/SURVIVED判定まで

本項では、筆者が開発したGo向けMutation Testingツールgomu^{注2}を用いて、そのしくみを

注2) <https://github.com/sivchari/gomu>

▼図2 gomouの処理フロー



解説します。gomouの処理フローは図2のとおりです。

ASTの走査とミュータント生成

ソースコードをプログラムが解析・操作しやすい木構造で表現したものをAST (Abstract Syntax Tree、抽象構文木) と呼びます。たとえば `a + b` というコードをASTに変換すると、「+という演算を、左辺aと右辺bに対して行う」という構造に分解されます。この+やa、bといった各要素をノードと呼び、これらが親子関係でつながることで木構造として形成されます。

GoにはASTを扱う標準パッケージとして `go/parser`、`go/ast`、`go/token` があります。`go/parser` でASTに変換し、`go/ast` の `ast.Inspect` で各ノードを走査できます。gomouではこれらの標準パッケージを使用して変換対象のノードを見つけるとミュータントを生成します。

たとえばリスト4のようなソースコードをgomouでは `go/parser` でASTに変換し、`ast.Inspect` で走査します。ノードが変換対象であれば、ミュータントを生成します (リスト5)。たとえば算術演算子のミュータントでは、+を見つけると-、*、/への書き換えを生成します (リスト6、図3)。

overlayによる安全な並列実行

gomouの特徴的な設計として、Goのoverlay機能^{注3)}の活用があります。overlayはGo 1.16

で導入された機能で、実際のファイルを変更せずに仮想的に別のファイルで置き換えるしくみです (リスト7)。

overlayを使うことで、もとのソースコードを直接変更せずに済むため途中で失敗しても安全です。ミュータントごとに独立したoverlay.jsonを持つためgoroutineで並列実行しても競合しません。

各ミュータントに対して `go test-overlay=overlay.json ./...` を実行し、テストが失敗すればKILLED、成功すればSURVIVEDと判定します (リスト8)。

▼リスト4 変換対象のソースコード

```
func add(a, b int) int {
    return a + b // 算術演算子+が変換対象ノード
}
```

▼リスト5 AST走査によるミュータント生成

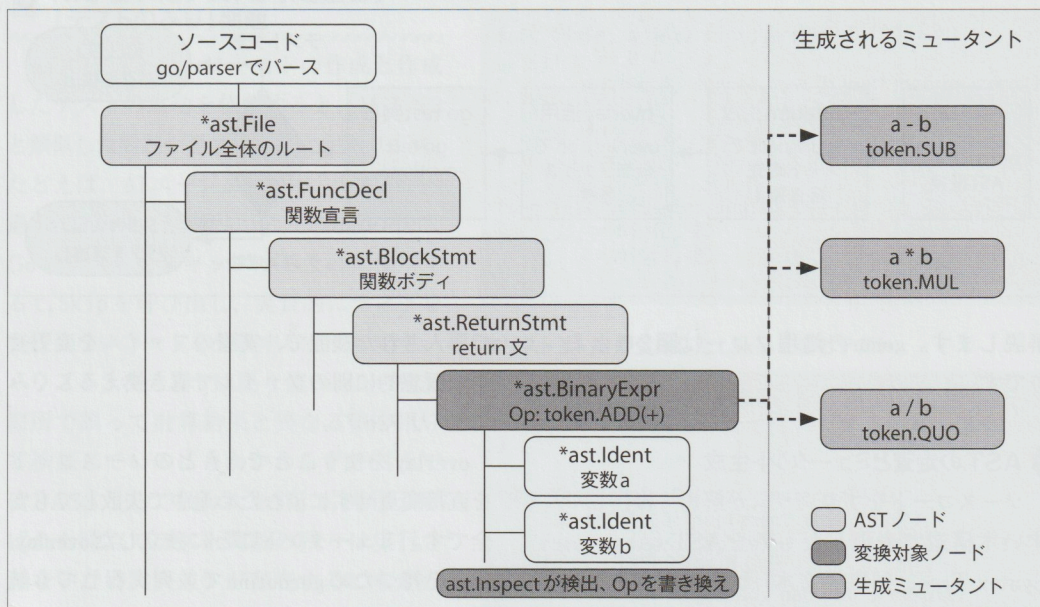
```
ast.Inspect(fileAST, func(n ast.Node) bool {
    for _, m := range e.mutators {
        if m.CanMutate(n) {
            mutants = m.Mutate(n, fset)
        }
    }
    return true
})
```

▼リスト6 算術演算子のミュータント生成

```
switch op {
case token.ADD: // +
    // -, *, /に変換
    return []token.Token{
        token.SUB, token.MUL, token.QUO,
    }
}
```

注3) https://pkg.go.dev/cmd/go#hdr-Overlay_files

▼図3 gomouにおけるASTの走査とミュータント生成



リスト2のTestDivに対してgomouを実行すると、たとえば a / b を $a * b$ に書き換えたミュータントがSURVIVEDになります。戻り値を検証していないため、演算子が変わってもテストは通ってしまうのです。このことから現在のTestDiv関数ではバグを検出できていないため、テストの品質が不十分であることがわかります。実際にSURVIVEDしたテストが見つかったあとは、対象のテストが検出できなかったケースを見て対応が必要かどうかを考えます。今回の例ではエラーの検証を省略したことが原因なのでエラーの検証をすることでKILLEDできるテストになります。

Mutation Testingを CIに組み込む

✓ 実行時間とのトレードオフ ——どこに適用するかを考える

Mutation Testingの最大の課題は実行時間です。ミュータントごとにテストスイート全体を実行するため、ミュータント数とテスト実行時間の積がそのまま合計実行時間になります。大規模なプロジェクトでは、全体に対してMutation

Testingを実行すると数時間かかることもあります。

そのため、「どこに適用するか」を考えることが重要です。すべてのコードに一律に適用するのではなく、ビジネスロジックの中核やバグが発生しやすい箇所に集中して適用するのが効果的です。gomouでは.gomignoreファイルで対象外のファイルやディレクトリを指定できます。

また、100% KILLEDを目指すべきではあ

▼リスト7 overlay.jsonの構造

```
{
  "Replace": {
    "/path/to/main.go": "/tmp/mutant_1/main.go"
  }
}
```

▼リスト8 KILLED/SURVIVEDの判定

```
cmd := exec.CommandContext(ctx,
  "go", "test",
  "-overlay="+mutCtx.OverlayPath,
  "...",
)
if err != nil {
  result.Status = mutation.StatusKilled
} else {
  result.Status = mutation.StatusSurvived
}
```


▼リスト9 gomuのGitHub Actions設定(.github/workflows/mutation.yml)

```

name: Mutation Testing
on:
  pull_request:
    branches: [main]

jobs:
  mutation:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@de0fac2e4500dabe0009e67214ff5f5447ce83dd # v6.0.2
        with:
          fetch-depth: 0
      - uses: actions/setup-go@4a3601121dd01d1626a1e23e37211e3254c1c06c # v6.4.0
        with:
          go-version-file: 'go.mod'
      - uses: sivchari/gomu@51fa62b8f3117cf2a65a49119a04af6fd001e4ce # main
        with:
          threshold: 80
          workers: 4
          timeout: 30
          incremental: true
          output: json
          upload-artifacts: true

```

りません。たとえば1 / 1を1 * 1に変換しても結果は同じ1であり、このミュータントがSURVIVEDであることには意味がありません。Goodhart's Lawはカバレッジだけでなく、ミューテーションスコアにも当てはまります。そのケースが本当に検証すべきものかを検討しながら、時にはIgnoreする判断も必要です。

☑ 差分のみに適用するインクリメンタルな運用とGitHub Actions設定例

実行時間の課題に対するgomuのアプローチが、インクリメンタル解析です。gomuはデフォルトで--incrementalが有効になっており、Gitの差分情報を利用して変更されたファイルのみをMutation Testingの対象にします。これにより、pull requestで変更したコードに対してのみMutation Testingを実行でき、実行時間を大幅に短縮できます。

「新しく書かれたコード、あるいはAIが生成したコードの品質をpull request単位で検証する」という運用が現実的です。gomuはGitHub Actionとして提供されており、リスト9のようにCIへ組み込みます。

設定のポイントは次のとおりです。

- fetch-depth: 0でインクリメンタル解析に必要な完全なGit履歴を取得する
- threshold: 80でミューテーションスコアが80%未満の場合にCIを失敗させる
- incremental: trueで差分ファイルのみを対象にする

gomuはCIモードで実行すると、pull requestにミューテーションスコアやSURVIVEDミュータントの詳細をコメントとして自動投稿します。レビュアーはpull request上でテストの弱点を直接確認でき、AIが生成したテストの品質を機械的に検証するワークフローが実現します。

AIがコードを書き、AIがテストを書く時代において、カバレッジだけではテストの品質を判断できません。Mutation Testingはテスト自体の品質を定量的に評価する手段です。gomuのインクリメンタル解析とGitHub Actionsとの統合を活用すれば、実行時間の課題を解決しながら、pull request単位でテストの品質を継続的に検証できます。テストをテストするしくみをCIに組み込むことが、AI時代の品質保証の鍵となるでしょう。SD